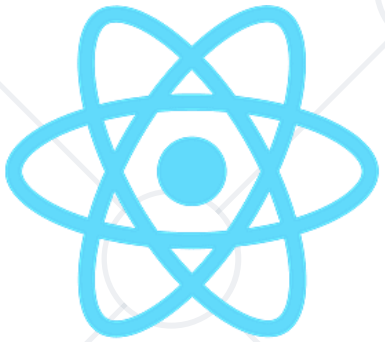


Layout, Styling & User Input

Build Native Mobile Apps with JavaScript and React



React Native

SoftUni Team
Technical Trainers



SoftUni



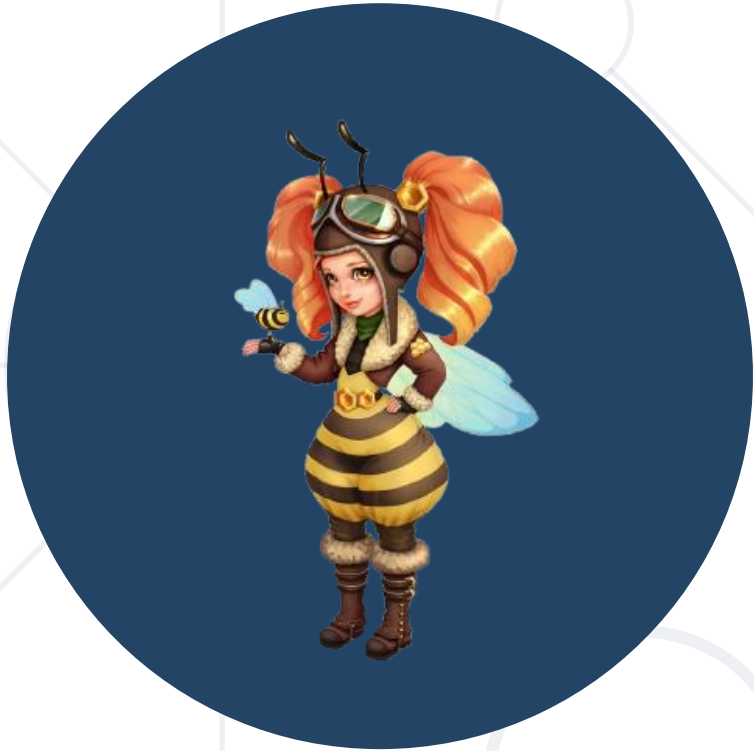
Software University

<https://softuni.bg>

Table of Contents

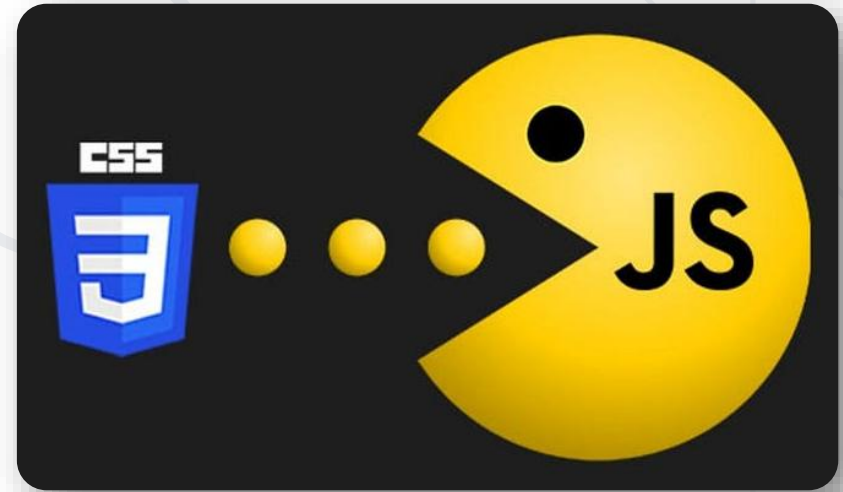
1. Styling in React Native
2. More Components
3. Layout
4. Device & Platform Aware Layouts
5. User Input
6. User Interaction





Styling in React Native

- **React Native styles are not CSS**
 - No HTML, No CSS, No DOM
 - No class names, No cascade, No selectors
- **React Native styling is JavaScript-based, not web-based.**



React Native styles

- Plain JavaScript Objects
- Inline styles
- Styling is component-driven
- Composition replaces cascade
- **Styles are predictable, scoped and explicit**



- Styles are **defined directly inside a component**:
 - Style properties are used
 - Good for **small or dynamic styles**
 - Supported **conditional styling**
 - Not ideal for **reuse or large styles**

```
<Text style={{ color: 'red', fontSize: 18 }}>  
  Hello React Native  
</Text>
```

- *Inline styles are **fine until you repeat yourself***

- **Combine** and **override styles**
 - Combine **multiple styles** together
 - Uses **style arrays**: `style={[...]}`
 - Supports overrides
 - Supports conditional styles
 - Improves **reuse and readability**
- *Combine styles for clean, reusable components*

- Create styles using **StyleSheet.create()**
 - Styles defined **outside components**
 - Improves **performance**
 - Encourages **reuse**
 - Easier to **maintain**
 - Cleaner component code
- *Use StyleSheet for structured, reusable styles*

- React Native uses a **simplified, device-independent** unit system
 - All numeric values are **unitless**
 - Values represent **density-independent pixels (dp)**
 - **Same style values** scale across **different screen sizes**
- No **px**, **em**, or **rem** units like on the web
- Layout **adapts automatically** to screen density
 - **%** is used for layout size only
- *Unitless values make layouts **consistent across devices***



More Components

- Controls the **appearance of the system status bar**
 - Sets **text color** (light or dark)
 - Controls **background color** (Android)
 - Can be **hidden** or **shown**
 - Configured **per screen** or **globally**
 - Helps match system UI with app design
- **Takeaway:** StatusBar helps your app feel native and polished

- Displays an image as a **background container**
 - Wraps other components **inside an image**
 - Commonly used for **headers** or **hero sections**
 - Supports **resize modes** (cover, contain)
 - Can be styled like a **regular View**
 - Useful for **layered UI designs**
- **Takeaway:** ImageBackground makes it easy to place content over images

- Enables **scrolling for content that exceeds the screen size**
 - Scrolls **vertically** or **horizontally**
 - Renders **all children at once**
 - Simple to use for small content
 - Supports **gestures** and **indicators**
 - Not ideal for long or large lists

Takeaway: Use **ScrollView** for **small, scrollable content**



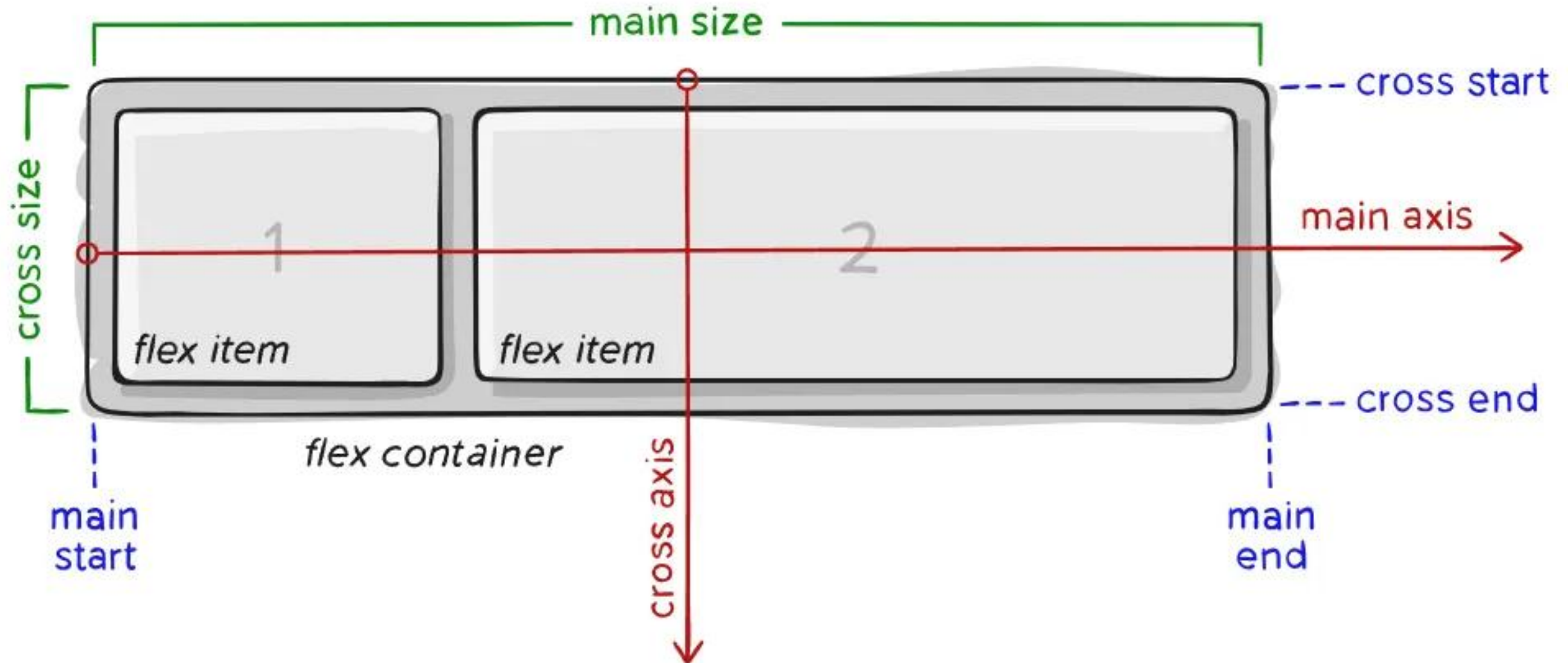
Layout

What is Flexbox?

- Flexbox is React Native's **primary layout system**
 - One-dimensional **layout model**
 - Arranges items in rows or columns
 - Handles **spacing** and **alignment**
 - Adapts to different screen sizes
 - Enabled by default in React Native
- Flexbox controls **how components are laid out**



Basic Terminology



- **Flex Container** – the **parent element**
- **Flex Item** – **direct children** of the container
- **Main Axis** – primary layout direction
- **Cross Axis** – perpendicular to main axis
- *Source: <https://css-tricks.com/snippets/css/a-guide-to-flexbox>*

- Properties that **control layout of children**
 - **flexDirection** – row or column
 - **flexWrap** – overflow mode
 - **justifyContent** – align along **main axis**
 - **alignItems** – align along **cross axis**
 - **alignContent** – align wrapped lines
 - **gap, rowGap, columnGap** – **spacing** between items
- *Container properties control alignment and spacing*

- Properties that **control individual items**
 - **alignSelf** - override container alignment
 - **flexGrow** - how items expand
 - **flexShrink** - how items shrink
 - **flexBasis** – initial size
 - **flex** - shorthand for **grow**, **shrink**, **basis**
- **Takeaway:** Item properties fine-tune individual layout behavior



Device & Platform Aware Layouts

- Two approaches to **handling different devices and platforms**
 - **Responsive** – fluid layout that scales smoothly
 - **Adaptive** – distinct layouts for specific sizes
- **Responsive** uses **flexible units and Flexbox**
- **Adaptive** uses **breakpoints or conditions**
- **React Native** can **support both approaches**
- *Responsive scales, adaptive switches layouts*

- Design layouts that **respond fluidly to different screen sizes**
 - No media queries
 - Use of **flexbox**
 - Use **percentage (%)** values
 - **Dimensions** – imperative API for **static screen size access**
 - **useWindowDimensions** –
reactive hook for **dynamic window size tracking**
- *Responsive layouts scale across devices*

- React Native is **cross-platform**, not platform-blind
 - **Platform.OS** - simple checks
 - **Platform.select()** - scalable platform logic
 - **SafeAreaView** - to avoid notches and system UI
 - **Orientation** - aware adaptation
 - **Performance** - aware adaptation
 - **Accessibility & User preferences**
- *Respect platform differences for better UX.*



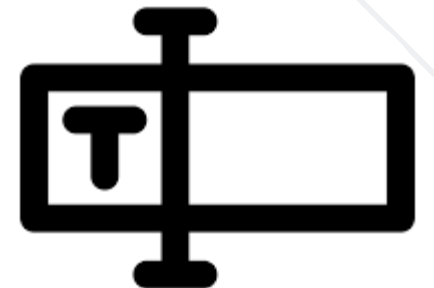
User Input

Switch

- A boolean input for **on/off** states
 - Used for **true** / **false** values
 - Controlled with **value** and **onValueChanged**
 - Native look on iOS and Android
 - Common for **settings** and **toggles**
 - Simple and lightweight input
- *Switch is best for binary choices*



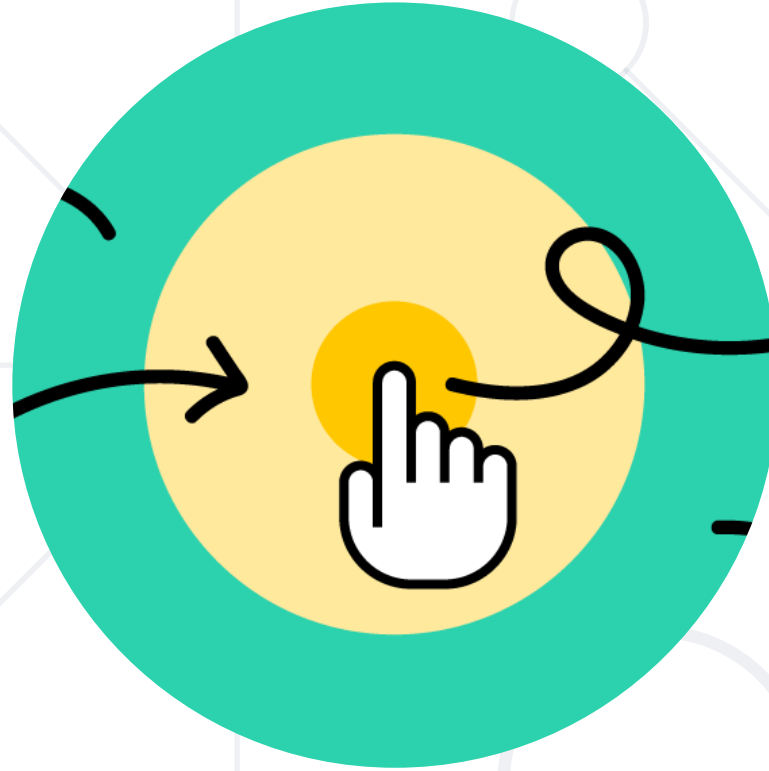
- Allows users to enter text
 - Supports **single-line** and **multiline input**
 - Controlled via **value** and **onChangeText**
 - **Keyboard type** can be customized
 - Supports **placeholders** and **auto-focus**
 - Core input for **forms** and **search**
- *TextInput is the foundation of user text input*



- Keyboard is **not part of the React Native layout**, it's an **OS-level overlay**
 - Appears when **TextInput** is focused
 - Can **overlap** or **resize layouts**
 - Platform-specific behavior
 - Can be **dismissed programmatically**
 - Important for form usability
- *Always design layouts with the keyboard in mind*



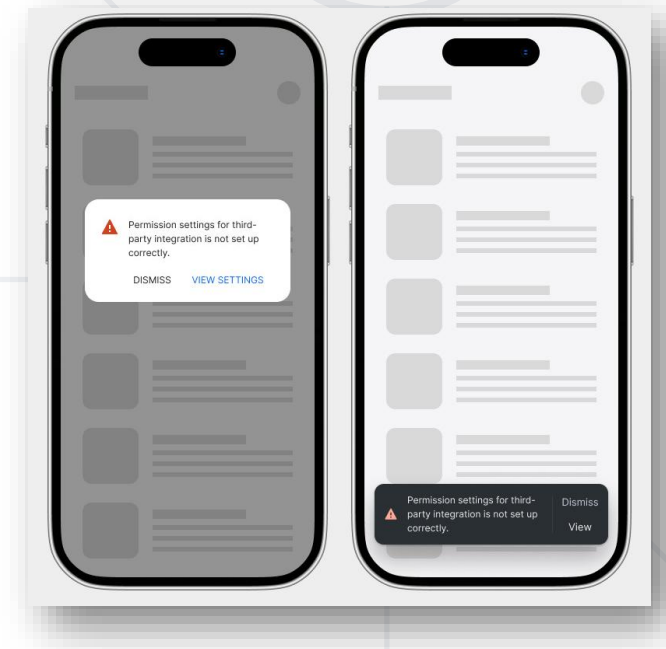
- Adjusts layout **when the keyboard appears**
 - Prevents inputs from being **hidden**
 - **Automatically shifts or resizes content**
 - Works with keyboard-aware layouts
- *KeyboardAvoidingView* **keeps inputs visible while typing**



User Interaction

User Feedback: Alerts & Modals

- Display important **messages** or **UI overlays**
 - Alert shows **system-native dialogs**
 - Used for **confirmations** and **warnings**
 - Blocks interaction until dismissed
 - **Modal** displays custom **full or partial screens**
 - Used for complex or custom interactions
- *Alerts inform, modals interact*



- Handle basic touch interactions
 - Respond to **user taps**
 - Provide visual feedback **on press**
 - *TouchableOpacity*
 - *TouchableHighlight*
 - *TouchableWithoutFeedback*
 - *TouchableNativeFeedback*
 - Being gradually **replaced by Pressable**
- *Touchables enable basic tap interactions*

- A modern, **flexible touch handler**
 - Detects press states (**pressed**, **hovered**, **focused**)
 - Supports **dynamic styling**
 - **More control** than touchables
 - Works **across** platforms
 - Recommended for new apps
- *Pressable is the future of touch handling*



Watch it happen

- React Native uses JavaScript-based styling, not HTML or CSS
- Layout is built with Flexbox, optimized for mobile screens
- Styles are explicit, scoped, and reusable via StyleSheet
- Layouts can be responsive and platform-aware
- User input is handled through native components
- Interaction is managed with modern touch APIs like Pressable



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg/>
- © Software University – <https://softuni.bg>

